الاحتراف الكامل

في مكتبات بايثون لعلم البيانات والتعلّم الآلي

الجزء الثاني عشر: Jupyter Notebook

الفصول ٧ - ١١: الأمثلة العملية، الأداء، التصحيح، أفضل الممارسات، والمشاريع المصغّرة

مرجع احترافي للمبرمجين والمهندسين وعلماء البيانات

جدول المحتويات

جدول المحتويات 2
الفصل السابع: الأمثلة العملية 4
٧.١ المستوى المبتدئ 4
مثال ١: دفتر استكشاف بيانات أولي منظّم 4
٧.٢ المستوى المتوسط 4
مثال ٢: استخدام Widgets لاستكشاف تفاعلي لفلاتر بيانات 4
٧.٣ المستوى الاحترافي 4
مثال ٣: دفتر تقرير قابل لإعادة التنفيذ بالكامل (Reproducible) 4
٧.٤ مثال من مشروع حقيقي: دفتر تدريب نموذج مع تتبع تجارب بسيط 5
تمارين عملية 5
حلول مقترحة (تلميحات) 6
الفصل الثامن: الأداء 7
٨.١ استهلاك ذاكرة Kernel التراكمي 7
٨.٢ قياس زمن التنفيذ داخل الدفتر 7
٨.٣ تجنّب إعادة تحميل بيانات ضخمة بلا داعٍ 7
أفضل الممارسات 7
الأخطاء الشائعة 7
الفصل التاسع: التصحيح 8
٩.١ تشخيص "الحالة الخفية" بشكل ممنهج 8
٩.٢ فحص حالة Kernel الحالية 8
٩.٣ مشاكل شائعة خاصة بالدفاتر 8
الأخطاء الشائعة 8
الأسئلة الشائعة 8
الفصل العاشر: أفضل الممارسات 9
١٠.١ عادة Restart & Run All الدورية 9
١٠.٢ فصل الاستكشاف عن الكود القابل لإعادة الاستخدام 9
١٠.٣ تسمية وترقيم الدفاتر 9
١٠.٤ التحكم بالإصدارات (Version Control) 9
أفضل الممارسات 9
الفصل الحادي عشر: مشاريع مصغّرة 10
مشروع ١: قالب دفتر استكشاف قابل لإعادة الاستخدام 10
مشروع ٢: أداة استكشاف تفاعلية شاملة بـ Widgets 10
مشروع ٣: مولّد تقرير HTML آلي من دفتر 11

تمارين عملية 11
حلول مقترحة (تلميحات) 11
مراجع وروابط رسمية 11
خاتمة Jupyter والمكتبات الست 11

الفصل السابع: الأمثلة العملية

٧.١ المستوى المبتدئ

مثال ١: دفتر استكشاف بيانات أولي منظّم

```
# الخلية 1 (Markdown): ## استكشاف بيانات المبيعات

# الخلية 2: الاستيرادات
import pandas as pd
import matplotlib.pyplot as plt

# الخلية 3: تحميل البيانات
df = pd.read_csv('sales.csv')
df.head()

# الخلية 4: نظرة سريعة
df.info()
df.describe()
```

شرح البنية: كل خلية لها مسؤولية واحدة واضحة (استيراد، تحميل، فحص)، وهذا النمط التدريجي هو جوهر الاستكشاف التفاعلي — رؤية النتيجة فورًا بعد كل خطوة قبل الانتقال للتالية.

٧.٢ المستوى المتوسط

مثال ٢: استخدام Widgets لاستكشاف تفاعلي لفلاتر بيانات

```
from ipywidgets import interact, Dropdown

@interact(city=Dropdown(options=df['city'].unique()))
def show_city_data(city):
    filtered = df[df['city'] == city]
    display(filtered.describe())
    filtered['sales'].hist(bins=20)
    plt.title(f'توزيع المبيعات - {city}')
    plt.show()
```

شرح الكود: هذا نمط استكشاف قوي جدًا داخل Jupyter تحديدًا — تغيير القيمة في القائمة المنسدلة يُعيد تنفيذ الدالة كاملة ويُحدّث الجدول والرسم فورًا، دون كتابة أي كود HTML/JavaScript.

٧.٣ المستوى الاحترافي

مثال ٣: دفتر تقرير قابل لإعادة التنفيذ بالكامل (Reproducible)

```
# الخلية 1 (Markdown): # تقرير أداء المبيعات – يناير 2026

# الخلية 2: إعدادات قابلة للتعديل في مكان واحد
REPORT_MONTH = '2026-01'
DATA_PATH = 'data/processed/sales_clean.parquet'

# الخلية 3
```

```
import pandas as pd
from src.reporting import build_summary, plot_trend   # منطق مستقر منقول لملف .py

# الخلية 4
df = pd.read_parquet(DATA_PATH)
summary = build_summary(df, month=REPORT_MONTH)
summary

# الخلية 5
plot_trend(df, month=REPORT_MONTH)
```

شرح الكود: فصل الإعدادات القابلة للتغيير (REPORT_MONTH) في خلية مبكرة واضحة يجعل إعادة توليد التقرير لشهر مختلف بتعديل سطر واحد فقط ثم Restart & Run All — وهذا هو معيار 'قابلية إعادة الإنتاج' في سياق الدفاتر الاحترافية.

٧.٤ مثال من مشروع حقيقي: دفتر تدريب نموذج مع تتبع تجارب بسيط

```
import json, datetime
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

params = {'C': 1.0, 'max_iter': 200}
model = LogisticRegression(**params)
model.fit(X_train, y_train)
acc = accuracy_score(y_test, model.predict(X_test))

log_entry = {
    'timestamp': datetime.datetime.now().isoformat(),
    'params': params,
    'accuracy': acc,
}

with open('experiments.jsonl', 'a') as f:
    f.write(json.dumps(log_entry) + '\n')

print(log_entry)
```

شرح الكود: تسجيل كل تجربة (المعاملات والنتيجة) في ملف سجل تراكمي بسيط هو أبسط شكل من أشكال 'تتبع التجارب' (Experiment Tracking)، ويحل مشكلة شائعة جدًا في دفاتر التعلّم الآلي: نسيان أي تركيبة معاملات أعطت أفضل نتيجة بعد عشرات التجارب المتفرقة.

تمارين عملية

١. أنشئ دفتر استكشاف لبيانات من اختيارك بخلايا منظّمة: استيراد، تحميل، فحص، رسم أولي.

٢. أضف أداة interact@ لفلترة رسم بياني حسب عمود فئوي من بياناتك.

٣. طبّق نمط تسجيل التجارب البسيط (JSON lines) على تدريب نموذجين بمعاملات مختلفة وقارن النتائج المسجَّلة.

حلول مقترحة (تلميحات)

١. اتبع بنية مثال ١ بالضبط مع بياناتك الخاصة.

٢. استخدم @interact(category=Dropdown(options=df['category'].unique())).

٣. كرّر حلقة تدريب-تسجيل لمعاملين مختلفين، ثم اقرأ experiments.jsonl عبر pd.read_json(lines=True).

الفصل الثامن: الأداء

٨.١ استهلاك ذاكرة Kernel التراكمي

بما أن كل الخلايا تتشارك ذاكرة Kernel واحدة طوال الجلسة، فإن المتغيرات الكبيرة (DataFrames ضخمة، مصفوفات NumPy كبيرة) تبقى في الذاكرة حتى لو لم تعد مستخدَمة، ما لم تُحذَف صراحة أو يُعاد تشغيل Kernel. في جلسات عمل طويلة على بيانات كبيرة، هذا التراكم قد يستهلك ذاكرة الجهاز تدريجيًا حتى مع كود يبدو سليمًا.

```
import gc

del huge_dataframe     # حذف المرجع للمتغير
gc.collect()           # فرض تشغيل جامع القمامة لتحرير الذاكرة فعليًا
```

٨.٢ قياس زمن التنفيذ داخل الدفتر

```
%%time
result = expensive_computation(data)

%timeit -n 10 -r 3 quick_function(x)    # تحكم دقيق بعدد التكرارات والدورات
```

٨.٣ تجنّب إعادة تحميل بيانات ضخمة بلا داعٍ

```
# سيئ: كل تشغيل للخلية يُعيد القراءة من القرص من جديد
df = pd.read_csv('huge_file.csv')   # في خلية قد تُعاد كتابتها/تشغيلها كثيرًا أثناء التطوير

# أفضل: تحميل مرة واحدة في خلية منفصلة أعلى الدفتر، وعدم إعادة تشغيلها إلا عند الحاجة الفعلية
```

فصل خلية 'التحميل الثقيل' عن خلايا 'التجريب السريع' التي تُعاد كتابتها بكثرة أثناء التطوير هو ممارسة أداء بسيطة لكنها فعّالة جدًا، لأنها تستغل نموذج التنفيذ التفاعلي بدل معاداته.

أفضل الممارسات

- افصل خلية تحميل البيانات الثقيلة عن خلايا التجريب السريع المتكررة.
- احذف المتغيرات الضخمة صراحة (del) واستدعِ gc.collect() عند الانتهاء منها في جلسة طويلة.
- استخدم timeit% بدل time%% عند الحاجة لقياس إحصائي دقيق متكرر، وليس تشغيلاً واحدًا فقط.

الأخطاء الشائعة

- إعادة تحميل بيانات ضخمة من القرص في كل مرة أثناء تجربة تعديل بسيط في خلية لاحقة.
- افتراض أن حذف خلية من الواجهة يحرّر الذاكرة التي استخدمتها تلقائيًا، بينما المتغيرات تبقى حيّة حتى تُحذَف صراحة أو يُعاد تشغيل Kernel.

الفصل التاسع: التصحيح

٩.١ تشخيص "الحالة الخفية" بشكل ممنهج

الفحص الأهم والأكثر إفادة عند سلوك غريب في دفتر يبدو صحيحًا: Kernel → Restart & Run All. إذا نجح الدفتر بالكامل من الصفر، فالمشكلة كانت حالة خفية متراكمة من تنفيذ سابق. إذا فشل عند خلية معيّنة، فأنت الآن تعرف بالضبط أين المشكلة الحقيقية دون التباس ترتيب التنفيذ.

٩.٢ فحص حالة Kernel الحالية

```
%whos           # عرض كل المتغيرات الحالية، نوعها، وحجمها التقريبي

%reset -f       # مسح كل المتغيرات من الذاكرة دون إعادة تشغيل Kernel بالكامل
```

٩.٣ مشاكل شائعة خاصة بالدفاتر

المشكلة	السبب والحل
الرسم لا يظهر رغم عدم وجود خطأ	تأكد من %matplotlib inline أو أن آخر سطر في الخلية هو كائن رسم فعلي/Figure
Kernel عالق في حالة Busy دائمة	حلقة لا نهائية أو عملية تحميل ضخمة؛ استخدم Interrupt Kernel لإيقافها
ImportError لمكتبة مثبّتة فعليًا	الدفتر يعمل على Kernel/بيئة مختلفة عن البيئة التي ثُبّتت فيها المكتبة
ملف .ipynb يعرض تعارضات Git غريبة	طبيعة صيغة JSON مع Metadata وأرقام تنفيذ؛ استخدم nbdime بدل diff النصي العادي

```
# التحقق من بيئة/مسار بايثون النشطة فعليًا داخل الـ Kernel الحالي
import sys
print(sys.executable)
```

طباعة sys.executable هي أسرع طريقة للتأكد من أن الدفتر يعمل فعلاً داخل البيئة الافتراضية الصحيحة التي ثبّتَّ فيها مكتباتك، وهي الأداة التشخيصية الأولى عند أي ImportError غامض رغم تأكدك من التثبيت.

الأخطاء الشائعة

- افتراض أن ImportError يعني أن المكتبة غير مثبّتة إطلاقًا، بينما السبب الفعلي غالبًا Kernel خاطئ لا يشير لنفس بيئة pip المستخدمة في الطرفية.
- محاولة حل تعارض Git في ملف .ipynb يدويًا كنص عادي بدل استخدام أداة مخصصة مثل nbdime.

الأسئلة الشائعة

س: ما الفرق بين Restart وInterrupt؟

ج: Interrupt يوقف تنفيذ الخلية الحالية فقط مع الحفاظ على كل المتغيرات السابقة في الذاكرة، بينما Restart يمسح ذاكرة Kernel بالكامل ويبدأ من جديد تمامًا.

الفصل العاشر: أفضل الممارسات

١٠.١ عادة Restart & Run All الدورية

أهم عادة احترافية واحدة في العمل مع Jupyter: قبل مشاركة أي دفتر أو اعتباره "منتهيًا"، نفّذ Restart & Run All من الصفر وتأكد أنه يعمل بلا أخطاء بنفس الترتيب المرئي للخلايا تمامًا. هذا يضمن خلو الدفتر من أي حالة خفية غير مرئية قد تُربك أي شخص آخر يفتحه (بما في ذلك أنت نفسك بعد أسبوع).

١٠.٢ فصل الاستكشاف عن الكود القابل لإعادة الاستخدام

- دفتر الاستكشاف: حر، تجريبي، يمكن أن يحتوي محاولات فاشلة ومسارات جانبية.
- وحدة src/: فقط الكود المستقر المُختبَر الذي أثبت جدواه، مستورَد داخل الدفاتر.
- دفتر التقرير النهائي: منظّم، نظيف، معتمِد كليًا على وحدات src/ دون منطق معقّد مباشر داخله.

١٠.٣ تسمية وترقيم الدفاتر

```
01_data_exploration.ipynb
02_data_cleaning.ipynb
03_feature_engineering.ipynb
04_model_training.ipynb
05_final_report.ipynb
```

١٠.٤ التحكم بالإصدارات (Version Control)

- استخدم jupytext لمزامنة كل .ipynb مع نسخة .py مقابلة تُتابَع في Git بوضوح أكبر بكثير.
- امسح المخرجات الكبيرة (خاصة الصور) قبل commit إن لم تكن ضرورية، لتقليل حجم المستودع.
- أضف nbstripout كخطاف Git (Git Hook) لتنظيف المخرجات تلقائيًا عند كل commit.

```
pip install nbstripout
nbstripout --install   # يُفعَّل تلقائيًا كخطاف pre-commit في المستودع الحالي
```

أفضل الممارسات

- نفّذ Restart & Run All قبل اعتبار أي دفتر جاهزًا للمشاركة.
- استخرج المنطق المستقر إلى وحدات .py بمجرد ثباته، ولا تتركه متكررًا في الدفاتر.
- استخدم jupytext وnbstripout لتحسين تجربة Git مع الدفاتر.

الفصل الحادي عشر: مشاريع مصغّرة

مشروع ١: قالب دفتر استكشاف قابل لإعادة الاستخدام

```
# الخلية 1 (Markdown): # قالب استكشاف بيانات قياسي

# الخلية 2: الإعدادات
DATA_PATH = 'data/raw/dataset.csv'
RANDOM_STATE = 42

# الخلية 3: الاستيرادات
import pandas as pd, numpy as np, matplotlib.pyplot as plt
pd.set_option('display.max_columns', 50)

# الخلية 4 (Markdown): ## 1. تحميل البيانات
# الخلية 5
df = pd.read_csv(DATA_PATH)
df.shape, df.dtypes

# الخلية 6 (Markdown): ## 2. جودة البيانات
# الخلية 7
df.isna().sum().sort_values(ascending=False).head(10)

# الخلية 8 (Markdown): ## 3. نظرة إحصائية سريعة
# الخلية 9
df.describe(include='all').T
```

يطبّق هذا المشروع: قالب منظّم بعناوين Markdown واضحة يمكن إعادة استخدامه كنقطة بداية لأي مشروع استكشاف بيانات جديد، مع فصل الإعدادات في خلية مبكرة قابلة للتعديل بسهولة.

مشروع ٢: أداة استكشاف تفاعلية شاملة بـ Widgets

```
from ipywidgets import interact, widgets

numeric_cols = df.select_dtypes('number').columns.tolist()

@interact(column=widgets.Dropdown(options=numeric_cols), bins=(5, 50, 5))
def explore_column(column, bins=20):
    fig, axes = plt.subplots(1, 2, figsize=(10, 4))
    df[column].hist(bins=bins, ax=axes[0])
    axes[0].set_title(f'توزيع {column}')
    df.boxplot(column=column, ax=axes[1])
    axes[1].set_title(f'كشف القيم الشاذة - {column}')
    plt.tight_layout()
    plt.show()
    print(df[column].describe())
```

يطبّق هذا المشروع: أداة استكشاف عامة تعمل مع أي عمود عددي في أي DataFrame تُمرَّر إليه، تجمع بين Widgets وMatplotlib وPandas في تجربة استكشاف تفاعلية واحدة سريعة جدًا مقارنة بكتابة كود منفصل لكل عمود يدويًا.

مشروع ٣: مولّد تقرير HTML آلي من دفتر

```
# بناء التقرير الكامل بخلايا منظمة: داخل الدفتر (كود + Markdown + رسوم)
# ثم من الطرفية بعد الانتهاء:

       jupyter nbconvert --to html --no-input report.ipynb --output final_report.html
```

يطبّق هذا المشروع: معامل --no-input في nbconvert يُنتج تقرير HTML نظيفًا يُخفي كود بايثون تمامًا ويُبقي فقط النصوص التوضيحية والرسوم والجداول، وهو الأسلوب المثالي لمشاركة نتائج تحليل مع جمهور غير تقني (إدارة، عملاء) دون إرباكهم بالكود.

تمارين عملية

١. استخدم قالب مشروع ١ على أي بيانات لديك وأكمله بأقسام إضافية (تحليل الارتباطات، القيم الفريدة للأعمدة الفئوية).

٢. وسّع أداة الاستكشاف التفاعلي في مشروع ٢ لتشمل أيضًا الأعمدة الفئوية عبر value_counts ورسم أعمدة.

٣. ولّد تقرير HTML نظيفًا بلا كود من أي دفتر لديك عبر nbconvert --no-input.

حلول مقترحة (تلميحات)

١. أضف خلية df.corr(numeric_only=True) مع تصويرها كخريطة حرارية عبر imshow.

٢. أضف فرعًا في الدالة: 'if df[column].dtype == 'object:
df[column].value_counts().plot(kind='bar').

٣. jupyter nbconvert --to html --no-input your_notebook.ipynb

مراجع وروابط رسمية

- التوثيق الرسمي: https://docs.jupyter.org/
- توثيق ipywidgets: https://ipywidgets.readthedocs.io/
- توثيق nbconvert: https://nbconvert.readthedocs.io/

خاتمة Jupyter والمكتبات الست

بهذا تُتم المرجع الكامل للمكتبة السادسة والأخيرة، Jupyter Notebook. عبر الأجزاء الاثني عشر من هذه السلسلة، أصبح لديك الآن فهم عميق لكل من NumPy وPandas وMatplotlib وPlotly وScikit-learn وJupyter: فلسفة تصميم كل واحدة، بنيتها الداخلية، دوالها الأساسية، أداؤها، تصحيحها، وأفضل ممارساتها. الملف الأخير في هذه السلسلة سيجمع كل هذه المكتبات معًا عبر فصل مقارنات شامل وWorkflow احترافي متكامل يوضّح كيف تتكامل جميعًا في مشروع علم بيانات حقيقي واحد من البداية للنهاية.